

Generowanie i estymacja parametrów dwuwymiarowych rozkładów α -stabilnych oraz wektorów sub-gaussowskich

Stanisław Olek

2026-06-16

Spis treści

1	Generator dwuwymiarowego rozkładu stabilnego	1
2	Generator dwuwymiarowego wektora losowego sub-gaussowskiego	2
3	Estymacja miary spektralnej Γ oraz parametru α	4
4	Estymacja funkcji charakterystycznej dla danych dwuwymiarowych	6
5	Estymacja miary kodiferencji τ	7
6	Wyznaczanie optymalnego wyboru parametru R w estymatorze miary spektralnej	8

1 Generator dwuwymiarowego rozkładu stabilnego

W celu wygenerowania próby (wektorów) z dwuwymiarowego rozkładu stabilnego dla zadanej dyskretnej miary spektralnej oraz parametru α (rozkład $\mathbf{S}(\alpha, \beta = 1, \gamma = 1, \delta = 0)$) zdefiniujmy generator:

```
def stable_generator(alpha, beta, gamma, delta):
    theta = np.random.uniform(-np.pi/2, np.pi/2, size=1)
    w = np.random.exponential(scale=1, size=1)
    if alpha != 1:
        theta_0 = np.arctan(beta * np.tan(np.pi * alpha / 2)) / alpha
        p1 = ((np.sin(alpha*(theta_0 + theta))) /
              ((np.cos(theta_0 * alpha)) * (np.cos(theta))))**(1/alpha)
        p2 = ((np.cos(alpha * theta_0 + (alpha - 1) * theta)) / w)**((1-alpha)/alpha)
        return gamma * p1 * p2 + delta
    else:
        p1 = (2/np.pi * (np.pi/2 + beta * theta) * np.tan(theta) -
              beta * np.log(np.pi/2 * w * np.cos(theta) / (np.pi/2 + beta * theta)))
        return gamma * p1 + (delta + beta * gamma * 2/np.pi * gamma * np.log(gamma))
```

```

def stable_twodim(s, gamma, n, alfa):
    Z = np.array([stable_generator(alpha=alfa, beta=1,
        gamma=1, delta=0) for _ in range(n)]).flatten()

    if alfa != 1:
        weights = (gamma**(1/alfa)) * Z
    else:
        weights = (gamma**(1/alfa)) * (Z + 2/np.pi * np.log(gamma))

    x_vector = np.sum(weights[:, np.newaxis] * s, axis=0)
    return x_vector

def generate_twodim_samples(s, gamma, n, alfa, num_samples=1000):
    samples = []
    for _ in range(num_samples):
        samples.append(stable_twodim(s, gamma, n, alfa))
    return np.array(samples)

```

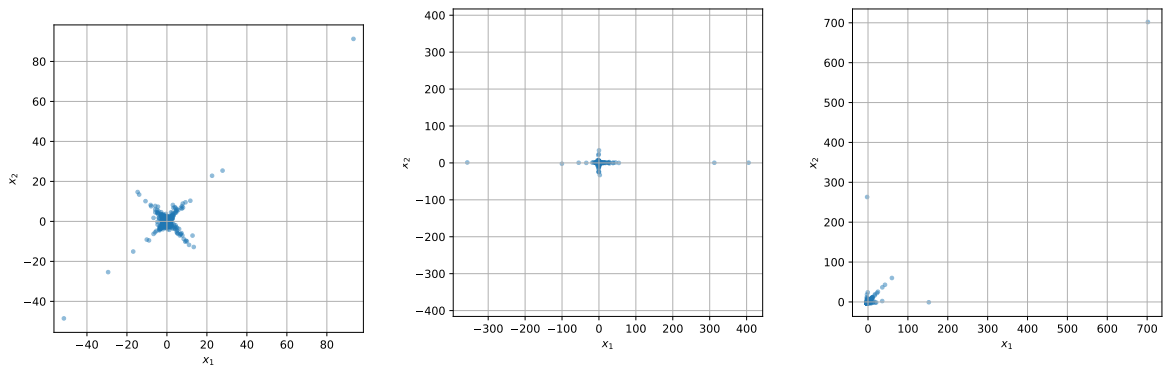
Wygenerujemy 3 próby (długości 1000) dla 3 różnych miar spektralnych i parametru $\alpha = 1.2$:

- symetryczny wektor stabilny, to znaczy taki, dla którego miara spektralna jest symetryczna względem początku układu współrzędnych; weźmy więc miarę spektralną w następujących 4 punktach: $(\sqrt{2}/2, \sqrt{2}/2)$, $(-\sqrt{2}/2, \sqrt{2}/2)$, $(-\sqrt{2}/2, -\sqrt{2}/2)$ i $(\sqrt{2}/2, -\sqrt{2}/2)$ z wagami 0.25 dla każdego z nich
- wektor stabilny o niezależnych składowych, to znaczy taki, dla którego miara spektralna jest dyskretna i skoncentrowana na osiach układu współrzędnych; weźmy więc miarę spektralną w następujących 4 punktach: $(1, 0)$, $(0, 1)$, $(-1, 0)$ i $(0, -1)$ z wagami 0.4, 0.1, 0.2 i 0.3 odpowiednio
- wektor stabilny, który nie jest symetryczny i nie ma niezależnych składowych, to znaczy taki, dla którego miara spektralna nie jest symetryczna względem początku układu współrzędnych i nie jest skoncentrowana na osiach układu współrzędnych; weźmy więc miarę spektralną w następujących 3 punktach $(1, 0)$, $(\sqrt{2}/2, \sqrt{2}/2)$ i $(0, 1)$ z wagami 0.2, 0.6 i 0.2 odpowiednio

Analizując wygenerowane próby (Rysunek 1), można zaobserwować wpływ dyskretnej miary spektralnej na zachowanie dwuwymiarowego wektora stabilnego. We wszystkich trzech przypadkach, ze względu na parametr $\alpha = 1.2 < 2$, dostrzegalne są grube ogony rozkładu poprzez skrajnie oddalone pojedyncze obserwacje. Dla symetrycznego wektora stabilnego (Rysunek 1 (a)) wartości odstające rozchodzą się we wszystkich czterech kierunkach wyznaczonych przez punkty miary spektralnej. W przypadku wektora o niezależnych składowych (Rysunek 1 (b)) punkty skupiają się wyłącznie na głównych osiach x_1 i x_2 , co potwierdza brak zależności między zmiennymi. Natomiast w trzecim wariancie (Rysunek 1 (c)) wprowadzenie niesymetrycznej miary spektralnej skutkuje brakiem niezależności oraz tym, że wartości ekstremalne są widoczne tylko w określonych kierunkach (w naszym przypadku są skupione głównie w pierwszej ćwiartce).

2 Generator dwuwymiarowego wektora losowego sub-gaussowskiego

W celu wygenerowania próby dwuwymiarowych wektorów losowych sub-gaussowskich, zdefiniujmy generator dla zadanego parametru α :



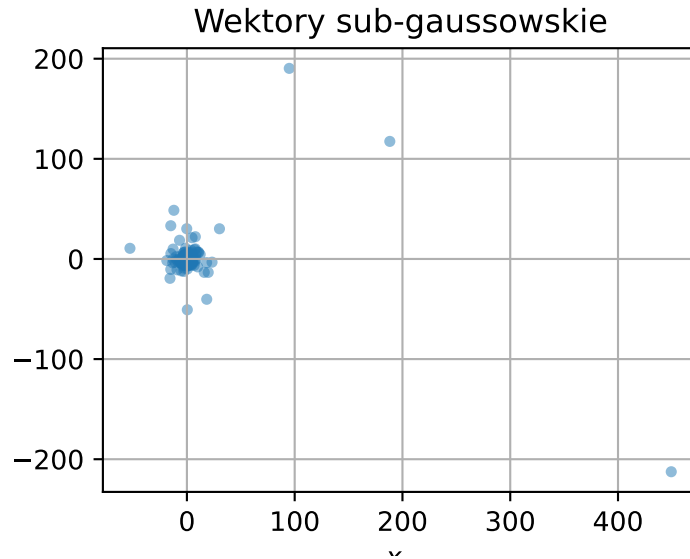
(a) symetryczny wektor stabilny (b) wektor o niezależnych składowych (c) niesymetryczny i o nie niezależnych składowych

Rysunek 1: Próby z dwuwymiarowego rozkładu stabilnego dla zadanych miar spektralnych i parametru $\alpha = 1.2$

```
def subgaussian_twodim(alfa):
    A = stable_generator(alpha=alfa/2, beta=1,
        delta=0, gamma=(np.cos((np.pi*alfa)/4))*(2/alfa))
    G = np.random.normal(0, 1, size=2)
    X = A**(1/2) * G
    return X

def generate_subgaussian_samples(alfa, num_samples=1000):
    samples = []
    for _ in range(num_samples):
        samples.append(subgaussian_twodim(alfa))
    return samples
```

Wygenerujmy próbę (długości 1000) z dwuwymiarowego rozkładu sub-gaussowskiego dlaadanego parametru $\alpha = 1.2$



Rysunek 2: Próba z dwuwymiarowego rozkładu sub-gaussowskiego dla zadanego parametru $\alpha = 1.2$

Jak możemy zauważyć (Rysunek 2), mimo że większość wygenerowanych punktów jest skoncentrowana wokół 0, na wykresie widać pojedyncze skrajnie oddalone obserwacje, co jest charakterystyczne dla rozkładów stabilnych dla parametru $\alpha < 2$ (własność grubych ogonów).

3 Estymacja miary spektralnej Γ oraz parametru α

W celu estymowania miary spektralnej Γ skorzystamy z estymatorów:

```
def estymujGamma(probka, fi1 = 0.0, fi2 = np.pi/2, R = 0.9):

    probkaX = probka[:, 0]
    probkaY = probka[:, 1]

    promienie = np.sqrt(probkaX ** 2 + probkaY ** 2)
    katy = np.atan2(probkaY, probkaX)

    indeks = int(np.size(promienie) * R)
    odciecie = np.sort(promienie)
    odciecie = odciecie[indeks]

    maska_promienie = promienie >= odciecie
    maska_katy = (katy >= fi1) & (katy <= fi2)

    gora = np.sum(maska_katy & maska_promienie)
    dol = np.sum(maska_promienie)

    return(gora/dol)
```

Estymator Γ opiera się na prostym zliczaniu kątów ekstremalnych obserwacji w zadanym stożku, co daje intuicyjną, geometryczną interpretację miary spektralnej jako rozkładu kierunków największych wartości. natomiast do estymatora parametru α mamy:

```
def estymujAlpha(probka, R = 0.8):

    probkaX = probka[:, 0]
    probkaY = probka[:, 1]

    promienie = np.sqrt(probkaX ** 2 + probkaY ** 2)
    promienie_sorted = np.sort(promienie)

    indeks = int(np.size(promienie) * R)
    ogon = promienie_sorted[indeks : ]

    empiryczny_ogon = np.array([(promienie.size - (indeks + j)) /
                                promienie.size for j in range(len(ogon))])

    log_ogon = np.log(ogon)
    log_ogon_empiryczny = np.log(empiryczny_ogon)

    regresja = linregress(log_ogon, log_ogon_empiryczny)

    alpha_h = -regresja.slope

    return(alpha_h)
```

Estymator α wykorzystuje najdalszą część próby i dopasowanie prostej w skali log - log dzięki czemu dobrze wychwytuje potęgową naturę ogona rozkładu stabilnego.

Estymacja α dla trzech próbek:

alpha (teoretyczne) = 1.2

alpha^_1 (symetryczny) = 1.4152170603879184

alpha^_2 (niezależne składowe) = 1.0243585427130457

alpha^_3 (niesymetryczny, zależny) = 1.1556725692288858

Następnie estymujemy Γ tylko dla pierwszej ćwiartki $(0, \frac{\pi}{2})$:

Szacowana masa miary spektralnej w pierwszej ćwiartce $(0, \pi/2)$:

symetryczny: 0.25

niezależne składowe: 0.31

niesymetryczny, zależny: 0.215

Dla wszystkich trzech prób estmowane wyniki α są bliskie wartości teoretycznej 1.2 co potwierdza, że ogon rozkładu promieni został poprawnie odtworzony przez symulacje oraz że estmator opary na regresji log-log zachowuje się stabilnie przy użytym promieniu R .

4 Estymacja funkcji charakterystycznej dla danych dwuwymiarowych

Mając do dyspozycji próbę danych $X^{(1)}, \dots, X^{(n)}$, gdzie $X^{(i)} = (X_1^{(i)}, X_2^{(i)})$, można estymować funkcję charakterystyczną za pomocą następującego estymatora:

$$\hat{\phi}(t) = \frac{1}{n} \sum_{i=1}^n e^{i(t_1 X_1^{(i)} + t_2 X_2^{(i)})}, \text{ gdzie } t = (t_1, t_2) \text{ jest dowolnym wektorem w } \mathbb{R}^2.$$

Estymator ten można rozłożyć na część rzeczywistą i urojoną:

$$\operatorname{Re}(\hat{\phi}(t)) = \frac{1}{n} \sum_{i=1}^n \cos(t_1 X_1^{(i)} + t_2 X_2^{(i)})$$

$$\operatorname{Im}(\hat{\phi}(t)) = \frac{1}{n} \sum_{i=1}^n \sin(t_1 X_1^{(i)} + t_2 X_2^{(i)})$$

Zaimplementujmy więc funkcję, która oblicza estymator funkcji charakterystycznej dla danej próby i punktu t :

```
def estimator_cf(sample, t):
    n = sample.shape[0]

    sum_re = 0.0
    sum_im = 0.0

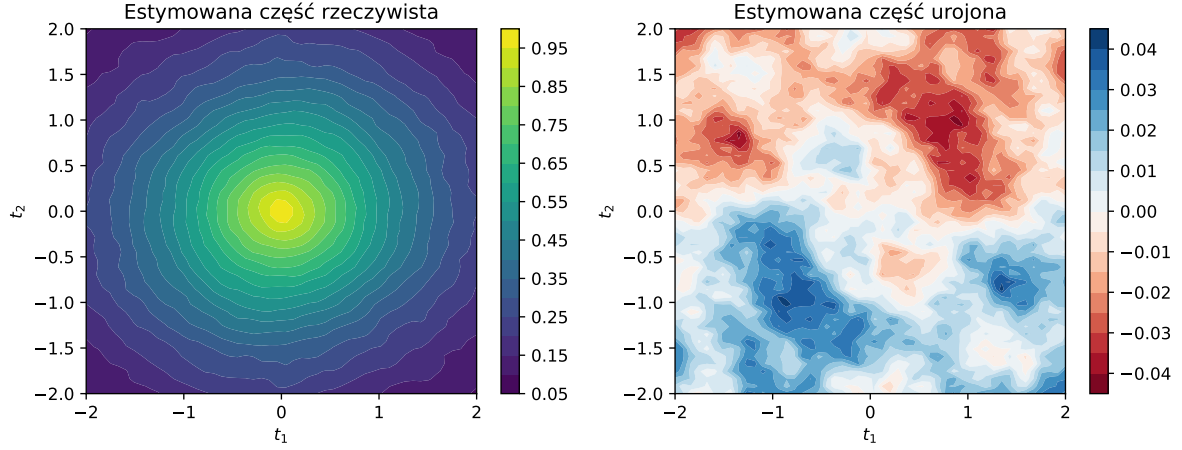
    t1 = t[0]
    t2 = t[1]

    for k in range(n):
        x1 = sample[k, 0]
        x2 = sample[k, 1]

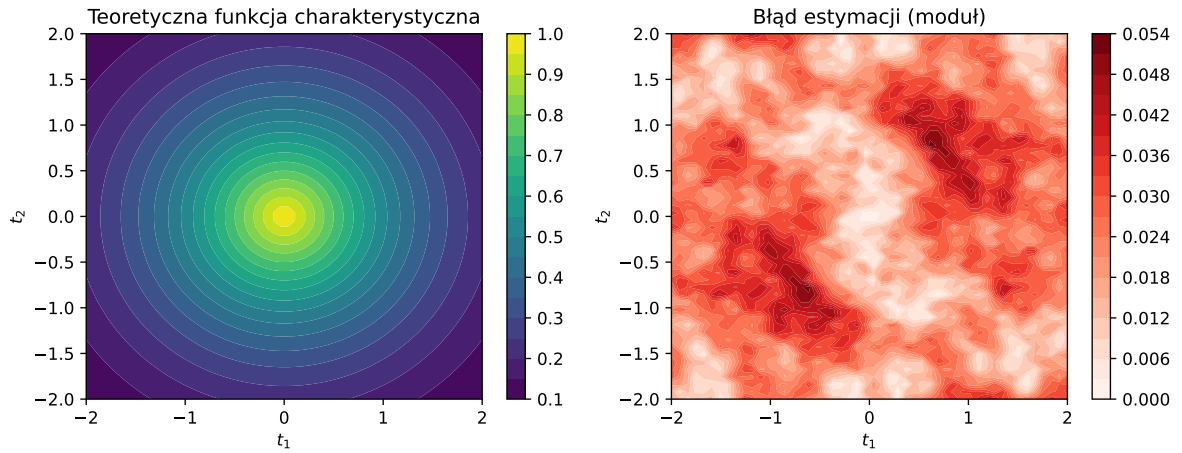
        dot_prod = (t1 * x1) + (t2 * x2)
        sum_re += np.cos(dot_prod)
        sum_im += np.sin(dot_prod)

    est_re = sum_re / n
    est_im = sum_im / n

    return est_re, est_im
```



(a) Część rzeczywista i urojona estymatora funkcji charakterystycznej



(b) Teoretyczna funkcja charakterystyczna i całkowity błąd estymacji

Rysunek 3: Estymacja funkcji charakterystycznej dla próby wektorów sub-gaussowskich z zadania 2 oraz porównanie jej z teoretyczną funkcją charakterystyczną

Teoretyczna funkcja charakterystyczna, dana wzorem $\phi(t) = \exp(-2^{-(\alpha/2)}|t|^\alpha)$, gdzie $|t| = \sqrt{t_1^2 + t_2^2}$, jest czysto rzeczywista (tj. $\text{Im}(\phi(t)) = 0$). Potwierdza to wykres estymowanej części urojonej (Rysunek 3 (a)), która jest bliska 0. Moduł błędu estymacji (Rysunek 3 (b)) przyjmuje bardzo małe wartości, co oznacza, że estymator $\hat{\phi}(t)$ jest zbieżny z wynikiem teoretycznym i potwierdza poprawność generatora wektora sub-gaussowskiego z zadania 2.

5 Estymacja miary kodiferencji τ

W celu estymacji miary kodiferencji $\tau(X_1, X_2)$ korzystamy z definicji estymatora podanego na ćwiczeniach. Zamplementowana funkcja przyjmuje jednowymiarowe próbki X, Y i zwraca oszacowanie $\tau(X, Y)$.

```
def codif(X, Y):
```

```

if np.size(X) != np.size(Y):
    raise ValueError
else:
    n = np.size(X)

p1 = np.log(1/n * np.real(np.sum( np.exp(1j*(X - Y)))))
p2 = np.log(1/n * np.real(np.sum( np.exp(1j*X))))
p3 = np.log(1/n * np.real(np.sum( np.exp(-1j*Y))))

return(p1 - p2 - p3)

def codif_from_sample(sample):

    X = sample[:, 0]
    Y = sample[:, 1]

    return codif(X, Y)

```

Wyniki konkretnych estymacji Kodyferencji:

Kodyferencja (symetryczny) = 0.4948351480955274

Kodyferencja (niezależne składowe) = -0.7325627666011775

Kodyferencja (niesymetryczny) = nan

Kodyferencja (alpha = 1.5) = 0.40815529091898917

Kodyferencja (alpha = 0.5) = 1.1787260754963693

Zauważmy, że dla przykładu z wektorem o niezależnych składowych estymowana kodyferencja nie jest równa zero co prawdopodobnie jest spowodowane konstrukcją generatora. Zwróćmy też uwagę, że przy niesymetrycznych miarach spektralnych estymator kodyferencji czasem zwraca NaN, co może wynikać z logarytmowania bardzo małych wartości i błędów numerycznych. Dodatkowo widać, że zmiana parametru α na 0.5 lub 1.5 istotnie wpływa na wartość kodyferencji, co pokazuje, że mierzy ona nie tylko zależność między składowymi, ale także wrażliwość na grubość ogonów rozkładu.

6 Wyznaczanie optymalnego wyboru parametru R w estymatorze miary spektralnej

Rozważmy problem wyznaczenia optymalnego wyboru parametru R w estymatorze miary spektralnej. W tym celu będzie nam potrzebny generator liczb o zadanej wartości $\alpha \in (0, 2]$ i mierze spektralnej Γ oraz estymator miary spektralnej $\hat{\Gamma}$.


```

# generator
rstable2d <- function(n, alpha, gamma_wagi, gamma_kierunki) {
  if (alpha <= 0 || alpha > 2)
    stop("Parametr 'alpha' musi być w przedziale (0,2].")
  if (abs(sum(gamma_wagi) - 1) > 1e-6)
    warning("Wagi w 'gamma_wagi' nie sumują się do 1.")
  m <- length(gamma_wagi)

  S <- matrix(NA, nrow = n, ncol = m)
  for (j in 1:m) {
    S[, j] <- rstable(n, alpha = alpha, beta = 0, gamma = 1, delta = 0)
  }

  wagi_skalowane <- gamma_wagi^(1/alpha)

  X <- matrix(0, nrow = n, ncol = 2)
  for (j in 1:m) {
    cos_theta <- cos(gamma_kierunki[j])
    sin_theta <- sin(gamma_kierunki[j])
    X[, 1] <- X[, 1] + wagi_skalowane[j] * S[, j] * cos_theta
    X[, 2] <- X[, 2] + wagi_skalowane[j] * S[, j] * sin_theta
  }
  return(X)
}

# estymator
est_gamma <- function(dane, R) {
  odleglosci <- sqrt(dane[,1]^2 + dane[,2]^2)
  kierunki <- atan2(dane[,2], dane[,1])
  maska_ogon <- odleglosci > R
  N <- sum(maska_ogon)
  if (N == 0) return(rep(0, 36))
  kierunki_ogon <- kierunki[maska_ogon]
  K <- 36
  granice <- seq(-pi, pi, length.out = K + 1)
  gamma <- rep(0, K)
  for (i in 1:K) {
    w_przedziale <- kierunki_ogon >= granice[i] & kierunki_ogon < granice[i+1]
    gamma[i] <- sum(w_przedziale) / N
  }
  return(gamma)
}

```

Funkcja *rstable2d* generuje n 2-wymiarowych wektorów o zadanym indeksie stabilności α i mierze spektralnej Γ . Funkcja *est_gamma* dla danych wejściowych oraz wartości R , wylicza kierunek dla każdej danej, dzieli kąt pełny na 36 równych części, zlicza jaką część obserwacji z danych zawiera się w poszczególnych częściach kąta pełnego i przypisuje tym częściom poszczególne masy. Pomysł na wyznaczenie optymalnego R jest taki, aby porównać estymator miary spektralnej $\hat{\Gamma}$ z rzeczywistą miarą spektralną Γ na podstawie błędu średniokwadratowego. Wartość R o

najmniejszym błędzie, będzie przyjętą przez nas optymalną wartością R . W tym celu musimy sprowadzić miarę spektralną do podobnej postaci jaką ma estymator.

```
#przeróbka by móc porównywać estymator z miarą spektralną
prawdziwa_gamma <- function(gamma_kierunki, gamma_wagi, granice) {
  K <- length(granice) - 1
  gamma_true <- rep(0, K)

  for (i in 1:K) {
    for (j in seq_along(gamma_kierunki)) {
      kat <- gamma_kierunki[j]
      kat <- ((kat + pi) %% (2*pi)) - pi

      if (kat >= granice[i] - 1e-10 && kat < granice[i+1] - 1e-10) {
        gamma_true[i] <- gamma_true[i] + gamma_wagi[j]
      }
    }
  }

  for (j in seq_along(gamma_kierunki)) {
    kat <- gamma_kierunki[j]
    kat <- ((kat + pi) %% (2*pi)) - pi
    if (abs(kat - pi) < 1e-10 || abs(kat + pi) < 1e-10) {
      gamma_true[K] <- gamma_true[K] + gamma_wagi[j]
    }
  }

  gamma_true <- gamma_true / sum(gamma_true)
  return(gamma_true)
}
```

Funkcja *prawdziwa_gamma* pełni taką rolę, mianowicie dla ustalonych mas i kierunków, które są ustalone dla miary spektralnej Γ oraz końców przedziałów podziału kąta pełnego, funkcja *prawdziwa_gamma* przypisuje przedziałom poszczególne masy dające się porównać z estymatorem.

```
# wybór optymalnego R
wybierz_R <- function(dane, gamma_kierunki, gamma_wagi,
                      kwantyle = seq(0.01, 0.99, by = 0.01)) {

  odleglosci <- sqrt(dane[,1]^2 + dane[,2]^2)
  progi_R <- quantile(odleglosci, kwantyle)

  K <- 36
  granice <- seq(-pi, pi, length.out = K + 1)
  gamma_true <- prawdziwa_gamma(gamma_kierunki, gamma_wagi, granice)

  bledy <- c()
  for (i in seq_along(progi_R)) {
    gamma_est <- est_gamma(dane, progi_R[i])
    if (sum(gamma_est) == 0) {
```

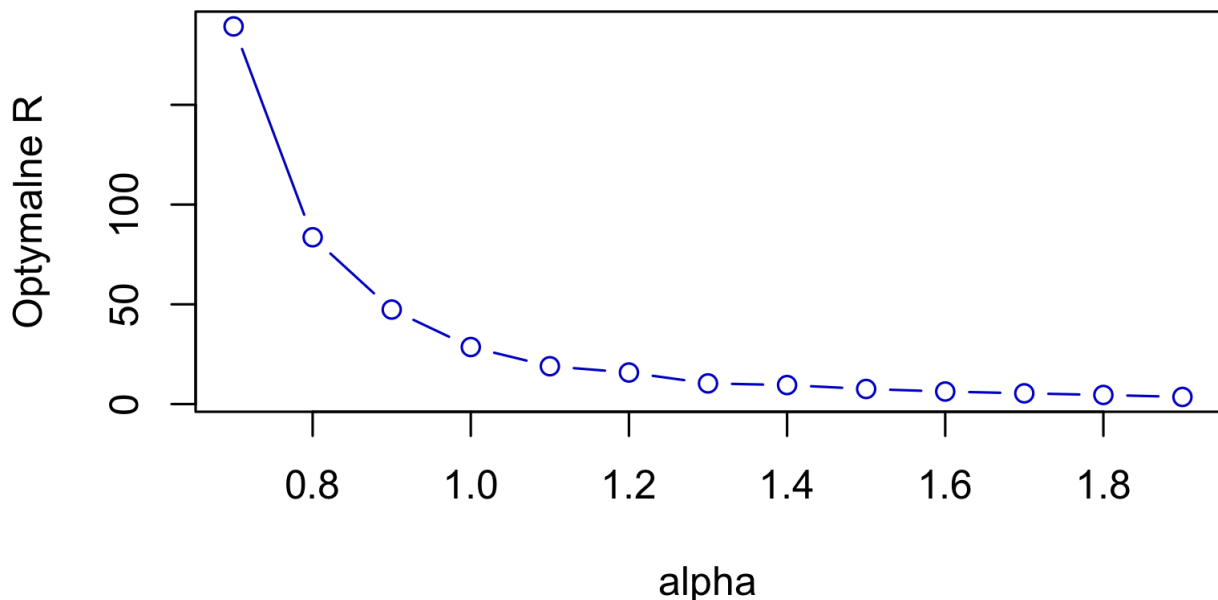
```

    bledy <- c(bledy, Inf)
  } else {
    mse <- mean((gamma_est - gamma_true)^2)
    bledy <- c(bledy, mse)
  }
}

idx <- which.min(bledy)
return(progi_R[idx])
}

```

Funkcja *wybierz_R* dla danych wejściowych oraz miary spektralnej Γ , z której wygenerowane zostały dane wybiera wartość R^* , dla której błąd średniokwadratowy jest najmniejszy. Jako kandydaci na R zostały rozpatrzyliśmy kwantyle próby, wybrane na zasadzie odległości od 0. Sprawdźmy zależność wyboru R^* od α oraz miary spektralnej Γ . W tym celu będziemy $m = 50$ razy generować próby rozmiaru $n = 10000$ o mierze spektralnej Γ_1 przypisującej wagę $\gamma_1 = 0.7$ punktowi $\mathbf{s}_1 = (1, 0)$ oraz wagę $\gamma_2 = 0.3$ punktowi $\mathbf{s}_2 = (0, 1)$, dla $\alpha \in \{0.7, 0.8, 0.9, \dots, 1.9\}$. Dla wybranej wartości α , za R^* przyjmiemy średnią R z m symulacji, a wszystko zwizualizujemy na wykresie.



Rysunek 4: Optymalne R w zależności od α

na podstawie wyresu 4, widzimy że R^* maleje wraz ze wzrostem α . Sprawdźmy teraz zależność od wyboru miary, w tym celu wybierzemy trzy miary:

- Γ_1 przypisująca wagę $\gamma_1 = 0.7$ punktowi $\mathbf{s}_1 = (1, 0)$ oraz wagę $\gamma_2 = 0.3$ punktowi $\mathbf{s}_2 = (0, 1)$
- Γ_2 przypisująca wagę $\gamma = 0.25$ do każdego z punktów: $\mathbf{s}_1 = (\sqrt{2}/2, \sqrt{2}/2)$, $\mathbf{s}_2 = (-\sqrt{2}/2, \sqrt{2}/2)$, $\mathbf{s}_3 = (-\sqrt{2}/2, -\sqrt{2}/2)$, $\mathbf{s}_4 = (\sqrt{2}/2, -\sqrt{2}/2)$
- Γ_3 przypisująca: wagę $\gamma_1 = 0.3$ punktowi $\mathbf{s}_1 = (1, 0)$, wagę $\gamma_2 = 0.4$ punktowi $\mathbf{s}_2 = (1/2, \sqrt{3}/2)$ oraz wagę $\gamma_3 = 0.3$ punktowi $\mathbf{s}_1 = (\sqrt{3}, -1/2)$.
- Γ_4 przypisująca wagę $\gamma = 0.5$ punktom $\mathbf{s}_1 = (0, 1)$ oraz $\mathbf{s}_2 = (0, -1)$

Następnie dla ustalonej wartości $\alpha_{\text{symulacja}} = 0.8$ zasymulujemy $m_1 = 100$ - krotnie, próbę rozmiaru $n = 10000$ dla wybranych miar spektralnych oraz użyjemy funkcji *wybierz_R* do wyboru R^* .

Gamma 1: 406.2117

Gamma 2: 285.6068

Gamma 3: 409.9768

Gamma 4: 0.01051013

Widzimy, że miara spektralna Γ z której pochodzą dane ma wpływ na wybór R^* , mianowicie dla Γ_2 wartość jest zdecydowanie niższa niż dla Γ_1 oraz Γ_3 , pewnie dlatego iż jest to miara symetryczna i równomierna, zatem potrzebujemy mniejszą ilość danych by kierunek danych się wykrył. Nie wspominając o Γ_4 , gdzie odpowiednie R , jest bliskie zeru.